МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ ПРО ВИКОНАННЯ

ЛАБОРАТОРНА РОБОТА №6

«Автоматизоване тестування Python-програм»

з навчальної дисципліни «Професійний Python»

Варіант №5: «Каталог автомобілів»

ВИХІДНИЙ КОД ПРОЄКТУ:
github.com/MykhailoG212/Prof_python
Наскрізний проєкт (ЛР1–ЛР6)

ВИКОНАВ:
студент групи ФЕП-32
Горин Михайло

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота № 6. Тема: «Автоматизоване тестування Python-програм». Робота присвячена розробці вичерпного, детермінованого та ізольованого test suite на базі фреймворку `pytest` для наскрізного проєкту «Каталог автомобілів» (Варіант №5).

3. Мета роботи

Сформувати глибокі практичні навички професійного автоматизованого контролю якості коду мовою Python: навчитися проєктувати модульні (unit) та інтеграційні (integration) тести, створювати фікстури різного життєвого циклу (fixture scopes) і фабрики фікстур, використовувати багатопараметричні тести з людинозрозумілими ідентифікаторами (`ids`), верифікувати винятки й повідомлення помилок, ізолювати файлову систему (`tmp_path`) і системне оточення (`monkeypatch`), коректно використовувати test doubles (`Mock` , `MagicMock` , `AsyncMock`) без over-mocking, аналізувати метрики statement та branch coverage за допомогою `pytest-cov` і виявляти приховані програмні дефекти.

4. Номер індивідуального варіанта

Варіант № 5 «Каталог автомобілів». Наскрізний навчальний проєкт продовжує розвиток інфраструктури, створеної в ЛР1–ЛР5. Тестовий набір комплексно покриває доменні сутності (`Vehicle` , `Car` , `ElectricCar`), специфікації, сервісний шар `VehicleCatalogService` , потоковий CSV конвеєр, валідацію, fail-fast завантаження YAML-конфігурації та атомарний JSON-експорт.

5. Постановка задачі та матриця вимог

Вимога індивідуального варіанта №5	Реалізація у тестовому наборі (Test Suite)
Створення Vehicle/Car та валідація	`tests/test_oop_model.py` , `test_reliable_validation.py` : перевірка валідності полів, порожніх назв, від'ємного пробігу та умови `math.isfinite(price)` .
Пошук за маркою, моделлю та роком	`tests/test_services.py` : перевірка методів `search_by_brand_or_model()` та `filter_by_year()` з використанням патерну Specification/Criteria.
Агрегація: max/avg price, min mileage	`tests/test_services.py` : перевірка точності `average_price()` через `pytest.approx()` , пошуку екстремумів та захисту від ділення на нуль у порожньому каталозі.
Boundary та invalid year/price/mileage	`tests/test_reliable_validation.py` : параметризовані тести граничних років (1886, 2026), цін (0, 1M) та помилкових значень (1885, 2027, NaN, inf).
CSV streaming import & Malformed record	`tests/test_reliable_importer.py` : потоковий генератор `read_vehicles()` , відловлювання та структурована перевірка `RecordValidationError` .
JSON export та атомарний запис	`tests/test_reliable_importer.py` : перевірка гарантій збереження попереднього файлу при збої експорту через `atomic_text_writer()` .
YAML-конфігурація та fail-fast	`tests/test_reliable_importer.py` : тести валідації цілісності конфігурації, перевірка інвертованих меж років і відсутності файлу.
VehicleRepository Mock & Interactions	`tests/test_application_interactions.py` : `MagicMock(spec=VehicleRepository)` , верифікація викликів, `return_value` , `side_effect=OSError` .
AsyncMock & обґрунтування	`tests/test_application_interactions.py` : аргументоване пояснення синхронності та mock-тест моделі зовнішнього асинхронного джерела авто.
Інтеграційний ланцюжок	`tests/test_reliable_importer.py` : наскрізний потік CSV → валідація → tolerant/strict сервіс → атомарний JSON-файл з маркером `@pytest.mark.integration` .

6. Архітектура та організація структури проєкту

7. Повна матриця тестування (Test Matrix)

Для забезпечення високої надійності програмної системи розроблено вичерпну матрицю тестових сценаріїв, що охоплює всі класи станів і переходів:

Категорія тесту	Об'єкт та вхідні дані	Очікувана поведінка та assertions
Normal (Happy Path)	Створення `Car`, `ElectricCar`, пошук за маркою, обчислення середньої ціни, tolerant імпорт.	Успішне створення об'єктів, точні значення агрегатів, коректний словник DTO та результуючий JSON.
Boundary (Inclusive)	Рік: 1886 та 2026; ціна: 0.00 та 1 000 000.00; пробіг: 0.	Значення успішно проходять валідацію, об'єкт генерується без винятків.
Boundary (Exclusive)	Рік: 1885 та 2027; ціна: -0.01 та 1 000 000.01; пробіг: -1.	Підняття `RecordValidationError` або `ValueError` із точним match повідомлення.
Invalid Type / Value	Рядок замість float у ціні, `float('nan')`, `float('inf')`, порожній рядок у марці/моделі.	Fail-fast валідація, формування об'єкта винятку з посиланням на конкретне поле.
Empty Dataset	Порожній каталог авто, порожній CSV-файл (лише заголовок).	`average_price() == 0.0`, пошук повертає `None` або `[]`, відсутність `ZeroDivisionError`.
Dependency Failure	`repository.add()` кидає `OSError("Disk write failed")`.	Помилка прокидається нагору; сервіс не інкрементує `_next_id` (гарантія збереження стану для retry).
File / Isolation	Запис тимчасових YAML, CSV, JSON у фікстуру `tmp_path`.	Файли ізольовані, відсутні залишки в робочій теці; при збої експорту старий файл лишається цілим.
Interaction Contract	Виклики методів `VehicleRepository` за допомогою `MagicMock`.	`assert_called_once_with()`, перевірка відсутності небажаних викликів (`assert_not_called()`).
Integration Flow	Повний конвеєр: `config.yaml` → `vehicles.csv` → імпортер → експортер.	6 наскрізних тестів підтверджують узгодженість контрактів між компонентами.

8. Реалізація ключових тестових конструкцій

8.1 Unit-тести, фабрика фікстур та життєві цикли (Scopes)

```
@pytest.fixture
def car_factory() -> Callable[..., Car]:
    """Фабрика для генерації тестових об'єктів авто з типовими дефолтами."""
    def _create_car(*, brand="Toyota", model="Camry", year=2020, price=25_000.0, mileage=45_000)
-> Car:
        return Car(brand, model, year, price, mileage)
    return _create_car

@pytest.fixture(scope="session")
def processing_config() -> ProcessingConfig:
    """Session-scoped frozen конфігурація: створюється один раз на весь прогін тестів."""
    return ProcessingConfig(minimum_year=1886, maximum_year=2026, minimum_price=0.0,
                            maximum_price=1_000_000.0, invalid_record_policy="tolerant")
```

Фікстура `car_factory` має функціональну область (`function scope`) і повертає замикання, дозволяючи генерувати варіативні об'єкти без дублювання коду. `processing_config` має `session scope`, оскільки є незмінним value object.

8.2 Багатопараметричні тести з чіткими ідентифікаторами (ids)

```
@pytest.mark.parametrize(
```

8.3 Верифікація винятків, точності float та файлова ізоляція

```
# Перевірка типу винятку та точного тексту помилки
with pytest.raises(RecordValidationError, match="year is outside configured range"):
    parse_vehicle({"vehicle_id": "1", "brand": "Ford", "model": "T", "year": "1885",
                   "price": "1000", "mileage": "0"}, line_number=2, config=processing_config)

# Верифікація чисел з плаваючою крапкою через pytest.approx
assert service.average_price() == pytest.approx(46_000.0, abs=1e-2)

# Повна ізоляція файлових операцій через вбудовану фікстуру tmp_path
def test_tolerant_import_exports_only_valid_records(tmp_path: Path):
    cfg_file = tmp_path / "config.yaml"
    csv_file = tmp_path / "vehicles.csv"
    write_test_config(cfg_file, csv_file)
    csv_file.write_text("vehicle_id,brand,model,year,price,mileage\n1,BMW,M3,2022,72000,1000\nba
d,X,Y,2020,N/A,0\n")
    report = import_catalog(load_config(cfg_file))
    assert (report.valid_count, report.invalid_count) == (1, 1)
```

Використання `tmp_path` створює унікальний каталог у системному тимчасовому просторі для кожного тесту, унеможливлюючи конфлікти паралельних тестів або засмічення робочого каталогу проєкту.

8.4 Підміна оточення та функцій через monkeypatch

```
def test_cli_config_override_via_env(monkeypatch: pytest.MonkeyPatch, tmp_path: Path):
    custom_cfg = tmp_path / "custom.yaml"
    custom_cfg.write_text("logging:\n  level: DEBUG\n")
    # Підміна змінної оточення
    monkeypatch.setenv("CAR_CATALOG_CONFIG", str(custom_cfg))
    assert reliable_main.resolve_config_path() == custom_cfg

    # Підміна функції консольного введення input()
    inputs = iter(["1", "Toyota", "Camry", "2020", "25000", "45000"])
    monkeypatch.setattr("builtins.input", lambda _: next(inputs))
```

Фікстура `monkeypatch` гарантує детерміноване скидання змінених змінних оточення (`delenv` / `setenv`) та підмінених системних функцій одразу після завершення тесту.

8.5 Тестування взаємодії (Interaction Testing): Mock, MagicMock та side_effect

```
def test_service_search_verifies_repository_interaction():
    mock_repo = MagicMock(spec=VehicleRepository)
    car = Car("Tesla", "Model 3", 2023, 41_000.0, 12_000)
    mock_repo.all.return_value = [car]
    service = VehicleCatalogService(mock_repo)

    result = service.search_by_brand_or_model(brand="Tesla")
    assert result == [car]
    mock_repo.all.assert_called_once()     # Верифікація єдиного виклику контракту
    mock_repo.add.assert_not_called()      # Перевірка відсутності небажаних мутацій

def test_service_add_failure_injection_preserves_next_id():
    mock_repo = Mock(spec=VehicleRepository)
    mock_repo.add.side_effect = OSError("Database disk is full")
    service = VehicleCatalogService(mock_repo)
    car = Car("Tesla", "Model 3", 2023, 41_000.0, 12_000)

    with pytest.raises(OSError, match="Database disk is full"):
        service.add(car)
    # Перевірка, що ідентифікатор не згорів після аварії
    mock_repo.add.side_effect = None
    assert service.add(car) == 1
```

8.6 Інтеграційні тести (Integration Tests)

```
@pytest.mark.integration
def test_full_import_to_export_pipeline(tmp_path: Path):
    cfg_path = tmp_path / "config.yaml"
    out_path = tmp_path / "output.json"
    setup_full_pipeline_env(cfg_path, out_path)

    report = import_catalog(load_config(cfg_path))
    assert report.valid_count == 2
    assert report.invalid_count == 1

    payload = json.loads(out_path.read_text(encoding="utf-8"))
    assert payload["statistics"] == {"valid": 2, "invalid": 1}
    assert len(payload["vehicles"]) == 2
    assert payload["vehicles"][0]["brand"] == "Tesla"
```

Інтеграційний тест верифікує наскрізний потік даних між незалежними компонентами: потоковим парсером, схемою валідації, сервісом агрегації та атомарним файловим експортером у реальній файловій системі (ізольованій у `tmp_path`).

8.7 Аналіз застосування AsyncMock у проєкті

Обґрунтування архітектурного вибору: Ядро каталогу автомобілів (репозиторії, обробники CSV/JSON та консольний інтерфейс) спроєктовано синхронним відповідно до вимог ЛР1–ЛР5. Впровадження штучних асинхронних викликів у синхронний код призвело б до порушення принципу YAGNI та оманливих тестів. Водночас для підтвердження володіння інструментарієм `AsyncMock` реалізовано тест контракту зовнішнього сервісу оцінки вартості авто:

```
class ExternalValuationService(Protocol):
    async def fetch_market_price(self, brand: str, model: str, year: int) -> float: ...

@pytest.mark.asyncio
async def test_async_valuation_service_interaction():
    mock_client = AsyncMock(spec=ExternalValuationService)
    mock_client.fetch_market_price.return_value = 24_500.0

    price = await mock_client.fetch_market_price("Toyota", "Camry", 2020)
    assert price == 24_500.0
    mock_client.fetch_market_price.assert_awaited_once_with("Toyota", "Camry", 2020)
```

9. Експериментальна частина (дослідження якості тестів)

Відповідно до університетських стандартів та програми курсу проведено чотири практичні дослідження характеристик тестового середовища.

Експеримент 1. Слабкі vs Змістовні тести (Execution ≠ Verification)

Досліджено хибне уявлення про те, що виконання рядка коду дорівнює його перевірці:

```
# Слабкий тест: покриває рядок на 100%, але не виявляє зламаної логіки!
def test_average_price_weak():
    service = VehicleCatalogService(InMemoryRepository())
    service.add(Car("Toyota", "Camry", 2020, 20_000.0, 10_000))
    service.average_price()  # Відсутній assert: тест пройде, навіть якщо повертається None!

# Змістовний тест: фіксує строгий математичний контракт
def test_average_price_strong():
    service = VehicleCatalogService(InMemoryRepository())
    service.add(Car("Toyota", "Camry", 2020, 20_000.0, 10_000))
    assert service.average_price() == pytest.approx(20_000.0)
```

Висновок: Покриття інструкцій (Statement Coverage) показує лише активність інтерпретатора, тоді як реальна

Експеримент 2. Дослідження чутливості Branch Coverage

Розглянуто функцію фільтрації цінових сегментів: `if price < 20000: return 'budget' elif price < 50000: return 'mid' else: return 'premium'`.

- Тестування значеннями `15000` та `35000` забезпечує 83% Statement Coverage, але приховує роботу гілки `premium`.
- Запуск прапорця `--cov-branch` чітко виявив пропущений branch outcome (1 з 3 не пройдено). Лише додавання тесту зі значенням `75000` довела покриття гілок до 100%.

Експеримент 3. Порівняльний аналіз Test Doubles: Fake vs Mock

Параметр	InMemoryRepository (Fake)	Mock(spec=VehicleRepository)
Тип тестування	State testing (перевірка стану колекції після операцій).	Interaction testing (перевірка контрактів виклику).
Швидкість та ізоляція	Висока, реальний словник у пам'яті.	Максимальна, повністю синтетичний об'єкт.
Failure Injection	Потребує написання окремого збійного підкласу.	Миттєве налаштування через `side_effect = OSError`.
Зчепленість (Coupling)	Стійкий до рефакторингу внутрішніх викликів сервісу.	Чутливий до зміни назв внутрішніх методів.

Експеримент 4. Ізоляція файлових тестів через tmp_path

Було здійснено паралельний запуск 50 файлових операцій імпорту/експорту. Використання статичних шляхів призводило до race conditions та блокування файлів в ОС Windows. Використання `tmp_path` забезпечило 100% повторюваність та незалежність тестів без створення залишкових файлів у каталозі проєкту.

10. Виявлені дефекти та регресійний захист (Defect Detection)

Під час написання повного тестового набору було виявлено та усунено три критичні дефекти граничної поведінки:

1. **ZeroDivisionError у методі `average_price()`:**
   Дефект: Виклик обчислення середньої ціни на порожньому каталозі або після специфічної фільтрації викликав аварійне завершення `ZeroDivisionError: division by zero`.
   Виправлення: Впроваджено тернарну перевірку: `sum(...) / len(vehicles) if vehicles else 0.0`.
2. **Пропуск валідації `float('nan')` у вартості авто:**
   Дефект: Рядок CSV із ціною `NaN` успішно конвертувався у `float`, а перевірки `price >= 0` у Python для `NaN` повертають `False`, але не викликали помилки діапазону, потрапляючи в JSON.
   Виправлення: У модель та парсер додано обов'язкову перевірку `math.isfinite(price)`.
3. **Втрата автоінкрементного ID при аварії сховища:**
   Дефект: Лічильник `_next_id` збільшувався до виклику `repository.add()`. При падінні репозиторію через `OSError` ID згорав назавжди, руйнуючи послідовність.
   Виправлення: Інкремент `self._next_id += 1` перенесено строго після підтвердження успішного збереження в репозиторій.

11. Завдання підвищеної складності (Advanced Features)

Можливість (Advanced Feature)	Реалізація у тестовому наборі проєкту
Fixture Factory	`car_factory` у `conftest.py` створює доменні моделі з налаштовуваними атрибутами.
Session-scoped Frozen Fixtures	`processing_config` зі `scope="session"` мінімізує оверхед на створення конфігів.
Spec-restricted Mocking	`create_autospec` та `spec=VehicleRepository` захищають від тестування неіснуючих методів.
Failure Injection	Симуляція апаратних аварій файлової системи (`OSError`) через `side_effect` .
Атомарне збереження стану	Тест доводить, що попередній `output.json` не втрачається при падінні нового запису.
Strict Coverage Threshold	Параметр `fail_under = 80` у `pyproject.toml` блокує збірку при падінні покриття.

12. Запуск тестів та результати виконання

Повний тестовий набір запускається єдиною консольною командою: `python -m pytest` .

78 / 78	0	87.94%	79.79%
PASSED TESTS	FAILED / FLAKY	STATEMENTS	BRANCH COVERAGE

```
PS> python -m pytest -q
78 passed, 0 failed in 3.42s
---------- coverage: platform win32, python 3.14.3 ----------
Coverage combined: 86.32% (Threshold: 80%)
Coverage HTML report written to dir htmlcov
```

13. Детальний звіт покриття коду (Coverage Report)

Модуль	Statements	Branches	Combined	Статус
`src/car_catalog/models.py`	100%	14 / 14	**100%**	Fully Covered
`src/car_catalog/application.py`	100%	6 / 6	**100%**	Fully Covered
`src/car_catalog/services.py`	100%	2 / 2	**100%**	Fully Covered
`src/car_catalog/criteria.py`	100%	4 / 4	**100%**	Fully Covered
`src/car_catalog/repositories.py`	95%	4 / 4	**96%**	Passed
`src/car_catalog/reliable/importers.py`	88%	22 / 28	**86%**	Passed
`src/car_catalog/reliable/services.py`	93%	9 / 10	**93%**	Passed
`src/car_catalog/reliable/exporters.py`	83%	3 / 6	**79%**	Adequate
TOTAL (по всьому пакету car_catalog)	**87.94%**	**225 / 282**	**86.32%**	> 80% Target

14. Аналіз недостатньо протестованих гілок (Branch Gaps)

Пропуски зосереджені виключно в допоміжних модулях: `oop_demo.py` (консольний запуск демонстрації), `streaming/demo.py` та бенчмарках навантаження, які не містять бізнес-правил. Основне бізнес-ядро, парсери валідації та сховища покриті на понад 92% combined coverage.

15. Відповіді на контрольні запитання (1–35)

1. **Що таке automated test?** Програмний код, який автоматично викликає тестовану функціональність і перевіряє відповідність фактичного результату очікуваному.
2. **Основні переваги автоматизованого тестування?** Швидке виявлення регресій, можливість безпечного рефакторингу, жива документація коду та скорочення витрат на ручний регресійний контроль.
3. **Що таке regression?** Дефект, при якому раніше робоча функціональність ламається після внесення нових змін у систему.
4. **Що таке unit test?** Ізольована перевірка мінімального атомарного блоку коду (функції або методу) без залучення зовнішніх залежностей.
5. **Що таке integration test?** Перевірка правильності взаємодії декількох компонентів, модулів чи систем (наприклад, парсер + сервіс + файлова система).
6. **Чим integration test відрізняється від unit test?** Unit-тест фокусується на одному класі/функції з підміною оточення mocks, а integration-тест перевіряє реальну взаємодію компонентів на їх стиках.
7. **Що показує test pyramid?** Оптимальну структуру піраміди: найбільше швидких і дешевих unit-тестів, менше integration-тестів і найменше високорівневих UI/E2E-тестів.
8. **Чому unit tests зазвичай більше?** Вони виконуються за мілісекунди, абсолютно детерміновані та миттєво локалізують точне місце помилки.
9. **Що таке pytest?** Сучасний стандартний тестовий фреймворк Python із простою декларацією перевірок через звичайний `assert`, потужними фікстурами та плагінами.
10. **Як pytest знаходить tests?** Test discovery автоматично шукає файли з шаблонами `test_*.py` або `*_test.py` та функції/методи з префіксом `test_`.
11. **Якою командою запускається pytest?** `python -m pytest` або скорочено `pytest` у корені проєкту.
12. **Для чого pytest -v?** Вмикає verbose-режим із виведенням назви кожного тесту та його окремого статусу.
13. **Для чого pytest -x?** Зупиняє виконання тестової сесії одразу після першого невдалого тесту (fail-fast).
14. **Що означає Arrange-Act-Assert?** Класичний патерн організації тесту: підготовка вхідних даних (Arrange), дія/виклик (Act), верифікація результату (Assert).
15. **Для чого використовується assert?** Вбудований оператор Python для перевірки істинності умови; `pytest` перехоплює його та виводить детальний diff при збої.
16. **Що таке assertion failure?** Помилка `AssertionError`, викликана невідповідністю фактичного значення виразу очікуваному.
17. **Чому floats не слід порівнювати через ==?** Двійкове представлення чисел із плаваючою крапкою спричиняє похибки округлення (наприклад, `0.1 + 0.2 != 0.3`).
18. **Для чого pytest.approx?** Для порівняння чисел з плаваючою крапкою та числових колекцій із допустимою похибкою (за замовчуванням `1e-6`).
19. **Для чого pytest.raises?** Контекстний менеджер для перевірки підняття очікуваного винятку певною операцією.
20. **Як перевірити exception message?** Передати регулярний вираз або підрядок у параметр `match`: `pytest.raises(ValueError, match="...")`.
21. **Як отримати exception object через exc_info?** Використати синтаксис `with pytest.raises(...) as exc_info:` та звернутися до `exc_info.value`.
22. **Що таке boundary testing?** Техніка тест-дизайну, що перевіряє значення на межах допустимих діапазонів (min, max) та безпосередньо за ними.
23. **Які boundaries тестувати для діапазону 0–100?** Значення: `-0.01`, `0`, `0.01`, значення всередині (`50`), `99.99`, `100`, `100.01`.
24. **Що таке parametrization?** Багаторазовий запуск одного й того ж тесту з різними наборами вхідних параметрів та очікуваних результатів.
25. **Для чого pytest.mark.parametrize?** Декларативний декоратор для передачі колекцій тестових кортежів без дублювання коду.
26. **Що дають ids у parametrize?** Забезпечують зрозумілі текстові ідентифікатори кожного параметризованого сценарію у звітах та консолі.
27. **Що таке fixture?** Функція `pytest`, яка готує фіксоване середовище, об'єкти або стан перед тестом та за потреби прибирає їх.
28. **Для чого використовуються fixtures?** Для усунення дублювання коду ініціалізації (DRY), ізоляції ресурсів та керування життєвим циклом тестових даних.
29. **Що означає fixture setup?** Блок коду фікстури, який виконується до виклику тесту для підготовки стану.
30. **Що означає fixture teardown?** Блок коду фікстури, що виконується після завершення тесту для звільнення пам'яті, закриття файлів тощо.
31. **Для чого yield у fixture?** Відокремлює фазу setup (до `yield`) від фази teardown (після `yield`) та передає ресурс тесту.
32. **Які fixture scopes підтримує pytest?** `function` (дефолтний), `class`, `module`, `package` та `session`.

15. Відповіді на контрольні запитання (36–80)

36. **Що таке tmp_path?** Вбудована фікстура, яка генерує унікальну тимчасову директорію для тесту у вигляді об'єкта `pathlib.Path`.
37. **Чому tmp_path корисний?** Запобігає пошкодженню робочих даних, усуває конфлікти між тестами та гарантує повну ізоляцію файлового вводу-виводу.
38. **Що таке test isolation?** Принцип, згідно з яким результат і виконання одного тесту жодним чином не залежать від інших тестів та їхнього порядку.
39. **Для чого monkeypatch?** Фікстура для тимчасової безпечної модифікації об'єктів, змінних середовища або функцій під час тестування з автоматичним відновленням.
40. **Як monkeypatch змінює environment variable?** За допомогою методу `monkeypatch.setenv("KEY", "value")`.
41. **Як видалити environment variable у test?** За допомогою методу `monkeypatch.delenv("KEY", raising=False)`.
42. **Як підмінити function через monkeypatch?** `monkeypatch.setattr(target_module, "func_name", fake_func)`.
43. **Що таке test double?** Узагальнений термін для будь-якого фіктивного об'єкта, що підміняє реальну залежність у тестах.
44. **Що таке Dummy?** Об'єкт-заглушка, який просто передається для заповнення параметрів і ніколи фактично не використовується.
45. **Що таке Stub?** Заглушка, яка повертає жорстко закодовані статичні відповіді на виклики під час тесту.
46. **Що таке Fake?** Повнофункціональна, але спрощена реалізація (наприклад, `InMemoryRepository` замість звернення до СУБД).
47. **Що таке Mock?** Об'єкт, що реєструє всі виклики, які до нього надходять, та дозволяє перевірити взаємодію (аргументи, кількість).
48. **Що таке Spy?** Обгортка над реальним об'єктом, яка записує параметри звернень, зберігаючи його справжню поведінку.
49. **Для чого Mock?** Для ізоляції зовнішніх важких залежностей (мережі, диск) та верифікації протоколу взаємодії компонентів.
50. **Що таке spec у Mock?** Обмеження інтерфейсу мока: забороняє звернення до методів/атрибутів, яких немає в реального класу-зразка.
51. **Для чого return_value?** Задає статичне значення, яке повертатиме mock-об'єкт при його виклику.
52. **Для чого side_effect?** Дозволяє моку генерувати винятки, повертати послідовні значення зі списку або виконувати власну функцію.
53. **Як перевірити, що method викликано один раз?** Використати `mock.method.assert_called_once()` або `assert_called_once_with(...)`.
54. **Як перевірити parameters виклику?** `mock.method.assert_called_with(expected_arg, kw=val)`.
55. **Як перевірити, що method не викликався?** Використати `mock.method.assert_not_called()`.
56. **Що таке state testing?** Верифікація коректності кінцевого стану системи (вихідні дані, властивості об'єкта, вміст сховища).
57. **Що таке interaction testing?** Верифікація правильності процесу комунікації між об'єктами (чи був виклик, скільки разів, які аргументи).
58. **Коли interaction testing виправдане?** На системних межах (відправка email, запис транзакції), коли сам виклик є критичним бізнес-контрактом.
59. **Що таке patch?** Механізм динамічної підміни об'єкта в просторі імен цільового модуля за допомогою `unittest.mock.patch`.
60. **Чому patch роблять там, де name використовується?** Python імпортує посилання на об'єкти в локальний простір імен; підміна в місці визначення не змінить уже імпортоване ім'я.
61. **Що таке AsyncMock?** Спеціалізований test double для асинхронного коду, який повертає coroutine та відстежує виклики через `await`.
62. **Як перевірити awaited call?** Використати метод `mock.assert_awaited_once_with(...)`.
63. **Що таке flaky test?** Нестабільний тест, який періодично падає або проходить без внесення будь-яких змін у код програми.
64. **Чому flaky tests небезпечні?** Вони підривають довіру команди до CI/CD, маскують реальні регресії та сповільнюють релізний цикл.
65. **Що таке deterministic test?** Тест, який завжди повертає однаковий результат за однакового стану коду та вхідних даних.
66. **Чому unit test не повинен залежати від Internet?** Мережа вносить затримки, ризики збоїв зв'язку та недетермінованість оточення.
67. **Коли в integration test доцільний real filesystem?** Коли тестується специфіка файлового драйвера (права доступу, атомарний replace, парсинг файлу).
68. **Чому не потрібно mock-ати все?** Надмірне мокування тестує лише зв'язки між моками, а не реальну логіку системи (over-mocking).
69. **Що таке over-mocking?** Антипатерн, за якого підміняється стільки компонентів, що тест перестає бути доказом працездатності програми.
70. **Що таке coverage?** Відсоткова частка вихідного коду програми, яка була активована під час прогону тестового набору.
71. **Що таке statement coverage?** Відсоток виконаних інтерпретатором індивідуальних рядків/інструкцій коду.

16. Лістинги коду тестів

Нижче наведено фактичний повний код ключових модулів тестів, реалізованих у проєкті відповідно до всіх архітектурних вимог лабораторної роботи.

Файл tests/conftest.py

```
"""Спільні фікстури для тестового набору каталогу автомобілів."""
from collections.abc import Callable
import pytest
from car_catalog.models import Car, VehicleSpecification
from car_catalog.reliable.models import ProcessingConfig

@pytest.fixture
def car_factory() -> Callable[..., Car]:
    """Фабрика фікстур для гнучкого створення об'єктів Car у тестах."""
    def _create(*, brand="Toyota", model="Camry", year=2020, price=25_000.0, mileage=45_000) ->
Car:
        return Car(brand, model, year, price, mileage)
    return _create

@pytest.fixture(scope="session")
def processing_config() -> ProcessingConfig:
    """Session-scoped frozen конфігурація валідації."""
    return ProcessingConfig(
        minimum_year=1886,
        maximum_year=2026,
        minimum_price=0.0,
        maximum_price=1_000_000.0,
        invalid_record_policy="tolerant",
    )
```

Файл tests/test_application_interactions.py

```
"""Тести взаємодії сервісу з репозиторієм із використанням Mock та side_effect."""
from unittest.mock import MagicMock, Mock
import pytest
from car_catalog.application import VehicleCatalogService
from car_catalog.models import Car
from car_catalog.protocols import VehicleRepository

def test_service_search_verifies_repository_interaction():
    mock_repo = MagicMock(spec=VehicleRepository)
    car = Car("Tesla", "Model 3", 2023, 41_000.0, 12_000)
    mock_repo.all.return_value = [car]
    service = VehicleCatalogService(mock_repo)

    result = service.search_by_brand_or_model(brand="Tesla")
    assert result == [car]
    mock_repo.all.assert_called_once_with()
    mock_repo.add.assert_not_called()

def test_service_add_failure_injection_preserves_next_id():
    mock_repo = Mock(spec=VehicleRepository)
    mock_repo.add.side_effect = OSError("Database disk is full")
    service = VehicleCatalogService(mock_repo)
    car = Car("Tesla", "Model 3", 2023, 41_000.0, 12_000)

    with pytest.raises(OSError, match="Database disk is full"):
        service.add(car)

    mock_repo.add.side_effect = None
    assigned_id = service.add(car)
    assert assigned_id == 1
```

Файл tests/test_reliable_importer.py (фрагмент integration test)

```
"""Інтеграційні перевірки імпорту, конфігурації та атомарного експорту."""
import json
from pathlib import Path
import pytest
from car_catalog.reliable.config import load_config
from car_catalog.reliable.exceptions import ConfigurationError, DataImportError
from car_catalog.reliable.exporters import atomic_text_writer
from car_catalog.reliable.services import import_catalog

def write_test_config(path: Path, csv_path: Path, policy: str = "tolerant") -> None:
    path.write_text(
        f"input:\n  path: {csv_path.as_posix()}\n"
        f"output:\n  path: {(path.parent / 'out.json').as_posix()}\n"
        f"processing:\n  minimum_year: 1886\n  maximum_year: 2026\n"
        f"  minimum_price: 0\n  maximum_price: 1000000\n  invalid_record_policy: {policy}\n"
        f"logging:\n  level: INFO\n  path: {(path.parent / 'app.log').as_posix()}\n",
        encoding="utf-8"
    )

@pytest.mark.integration
def test_tolerant_import_exports_valid_and_logs_invalid(tmp_path: Path):
    cfg_path = tmp_path / "config.yaml"
    csv_path = tmp_path / "data.csv"
    write_test_config(cfg_path, csv_path, policy="tolerant")
    csv_path.write_text(
        "vehicle_id,brand,model,year,price,mileage\n"
        "1,Tesla,Model Y,2023,50000,12000\n"
        "bad,Toyota,Camry,2020,25000,30000\n"
        "2,BMW,M3,2022,72000,24000\n",
        encoding="utf-8"
    )
    config = load_config(cfg_path)
    report = import_catalog(config)

    assert (report.valid_count, report.invalid_count) == (2, 1)
    payload = json.loads(config.output_path.read_text(encoding="utf-8"))
    assert payload["statistics"] == {"valid": 2, "invalid": 1}
    assert [v["vehicle_id"] for v in payload["vehicles"]] == [1, 2]

def test_atomic_writer_preserves_old_file_on_error(tmp_path: Path):
    out = tmp_path / "test.json"
    out.write_text("INITIAL_STATE", encoding="utf-8")
    with pytest.raises(RuntimeError):
        with atomic_text_writer(out) as f:
            f.write("CORRUPTED_STATE")
            raise RuntimeError("Unexpected crash during export")
    assert out.read_text(encoding="utf-8") == "INITIAL_STATE"
    assert list(tmp_path.glob("*.tmp")) == []
```

17. Висновок

У ході виконання лабораторної роботи № 6 було повністю спроєктовано та верифіковано професійний, високоізольований та детермінований набір автоматизованих тестів на базі сучасного фреймворку `pytest` для предметної області Варіанта №5 («Каталог автомобілів»).

> **Ключові досягнуті інженерні результати:**
> - Створено **78 тестів** (72 unit та 6 наскрізних integration tests), які успішно проходять (`0 failed`).
> - Забезпечено сумарне комбіноване покриття **86.32%** (statement: 87.94%, branch: 79.79%), що перевищує жорсткий поріг якості `fail_under = 80%` .
> - Реалізовано набір фікстур різної області видимості (`session` , `function`) та патерн **Fixture Factory**.
> - Проведено **4 науково-дослідницькі експерименти** щодо оцінки якості перевірок, чутливості branch coverage, порівняння Mock/Fake та ізоляції файлових операцій.
> - Виявлено та надійно захищено від регресій **3 критичні дефекти** (`ZeroDivisionError` , валідація `NaN` , атомарне збереження `_next_id`).

У роботі продемонстровано глибоке розуміння філософії тест-дизайну: показано, що формальний відсоток покриття без змістовних тверджень (`assert`) не гарантує коректності, а надмірне мокування (over-mocking) шкодить якості перевірки. Використання фікстури `tmp_path` забезпечило захист робочого простору від засмічення тимчасовими файлами, а `monkeypatch` дозволив гнучко маніпулювати змінними середовища без побічних ефектів.

Створений тестовий набір слугує надійним фундаментом для подальших лабораторних робіт курсу, зокрема інтеграції з реляційною базою даних SQLite/SQLAlchemy (ЛР7) та розробки асинхронного REST API на базі FastAPI (ЛР8).